

UNIVERSIDAD
PANAMERICANA

Artificial intelligence in videogames

2nd Partial Project

Shortest path search with Dijkstra's algorithm

Student:
Omar Alejandro Santos Ibarra

Team members:
Uriel Arturo Monarrez Cervantes
Jaime Rincón Burboa
Omar Alejandro Santos Ibarra

Professor:
Alfredo Emmanuel Garcia Falcon

Due date:
Friday October 23rd

1. Introduction

Finding the cheapest route between two points is one of the most common problems in videogames: an enemy chasing the player, a unit moving across a strategy map or a character navigating a level all need a path that is valid and, ideally, optimal. Dijkstra's algorithm is the classic solution for this problem when every connection has a non-negative cost (weight).

The objective of this project is to implement a shortest path search program using the Dijkstra algorithm. The program lets the user build a weighted directed graph, choose a start node and an end node, edit the weights of the connections, and see both the sequence of nodes and the total weight of the shortest path. The application was written in Python and its graphical interface was built with Tkinter.

2. Dijkstra algorithm explanation

2.1 General idea

Dijkstra's algorithm finds the path of minimum total weight from a start node to every other node of a graph (or to a specific end node) as long as no edge has a negative weight. It works in a greedy way: at every step it takes the unvisited node that is currently closest to the start, marks it as visited, and checks whether going through that node gives a shorter route to each of its neighbours. When a node is marked as visited, its distance is final and will never be improved.

2.2 Step by step

1. Assign a tentative distance to every node: 0 for the start node and infinity for all the others. Also keep, for each node, a reference to its predecessor (initially none).
2. Put all nodes in the set of unvisited nodes. The start node becomes the current node.
3. For the current node, visit each of its outgoing connections. Calculate the new distance to the neighbour as distance(current) + weight(current → neighbour).
4. If this new distance is smaller than the distance stored for the neighbour, update it and set the current node as the neighbour's predecessor (this is called relaxation).
5. Mark the current node as visited. It will not be processed again.
6. Select, among the unvisited nodes, the one with the lowest tentative distance and make it the new current node.
7. Repeat from step 3 until the end node is selected as the current node (or until no reachable unvisited node is left, in which case there is no path).
8. Rebuild the shortest path by starting at the end node and following the predecessors back to the start node, then reverse the sequence.

2.3 Pseudocode

```
function Dijkstra(graph, start, end):
    for each node v in graph:
```

```
        dist[v] = infinity
        prev[v] = none
    dist[start] = 0
    queue = priority queue containing (0, start)

    while queue is not empty:
        (d, u) = queue.extract_min()
        if d > dist[u]: continue          # outdated entry, skip
        if u == end: break
        for each (v, w) in neighbours(u):
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                prev[v] = u
                queue.insert((dist[v], v))

    path = []
    u = end
    while u is not none:
        path.insert_at_front(u)
        u = prev[u]
    return path, dist[end]
```

2.4 Complexity and limitations

Using a binary heap as priority queue (the approach used in this project through Python's heapq module), the algorithm runs in O((V + E) log V), where V is the number of nodes and E the number of connections. With a simple list or array to search for the minimum, the cost would be $O(V^2)$. The algorithm only guarantees a correct result when all weights are non-negative, which is why the program rejects negative weights when the user edits a connection.

2.5 How the algorithm maps to the code

The following table relates each element of the algorithm with the part of the program that implements it (all of it lives in the dijkstra() method of the Grafo class):

Algorithm element	Implementation in the code
Initial node / final node	Parameters etiqueta_inicio and etiqueta_fin of dijkstra(); in the UI they are stored in nodo_inicio and nodo_fin.
Tentative distance of each node	Dictionary distancias, initialized to float('inf') for every node and to 0 for the start node.
Predecessor of each node	Dictionary predecesores, initialized to None.
Priority queue / node with the lowest weight	List pq managed with heapq. heappop() always returns the entry with the smallest distance; entries are tuples (distance, label, node).

Algorithm element	Implementation in the code
Change a node from unvisited to visited	A node is considered visited the first time it is popped with its final distance. Stale entries are discarded with the condition if dist_actual > distancias[nodo_actual]: continue, so a node is never processed twice.
Update the current node	Each iteration of the while loop pops the lowest-weight entry from pq and assigns it to nodo_actual.
Calculate the weight of a node	For every neighbour: nueva_dist = dist_actual + peso. If it is lower than distancias[vecino], the distance and the predecessor are updated (relaxation) and the neighbour is pushed to pq.
Stop condition	if nodo_actual == fin: break.
Path reconstruction	A loop starts at the end node and follows predecesores backwards, inserting each node at the front of the list camino. If the distance of the end node is still infinity, the method returns an empty path and float('inf').

3. Description of data structures, classes, methods and attributes

The program is a single Python file organized in three classes: Nodo and Grafo (the model) and AplicacionDijkstra (the graphical interface). Python modules used: tkinter (interface), random (random initial weights), heapq (priority queue) and math (geometry used to draw the arrows).

3.1 Class Nodo

Represents a node of the graph.

Attributes

Attribute	Description
`etiqueta`	Unique label that identifies the node (1, 2, 3, …). It is the label shown inside the node in the UI.
gx	Column of the node in the grid (horizontal position).
gy	Row of the node in the grid (vertical position).

Methods

Method	Description
`__init__(etiqueta, grid_x, grid_y)`	Constructor. Stores the label and the grid position.
`__eq__(otro)`	Two nodes are equal if they have the same label.

Method	Description
`__hash__()`	Returns the hash of the label, so nodes can be used as dictionary keys.
`__repr__()`	Text representation of the node, for example Nodo(3).

3.2 Class Grafo

Represents a directed graph with weights. The connections of each node (its outgoing edges and their weights) are stored in the adjacency structure, so the Nodo class does not need to store them itself.

Attributes

Attribute	Description
`nodos`	Dictionary {label: Nodo} that gives direct access to every node by its label.
`adyacencia`	Adjacency dictionary {origin node: {destination node: weight}}. It represents every directed connection with its weight.

Methods

Method	Description
`agregar_nodo(etiqueta, grid_x, grid_y)`	Creates a node with the given label and grid position, registers it in nodos and creates its empty entry in adyacencia. If the label already exists, it returns the existing node.
`agregar_arista_dirigida(etiqueta_u, etiqueta_v, peso=None)`	Creates the directed connection u → v. If no weight is given, a random integer between 1 and 9 is assigned.
`actualizar_peso(etiqueta_u, etiqueta_v, nuevo_peso)`	Changes the weight of an existing connection u → v.
`dijkstra(etiqueta_inicio, etiqueta_fin)`	Runs Dijkstra's algorithm. Returns a tuple (path, total cost), where the path is a list of Nodo objects. Returns ([], infinity) if a node does not exist or if there is no path.

3.3 Class AplicacionDijkstra

Inherits from tk.Tk and builds the whole graphical interface: the initial screen, the canvas where the graph is drawn, the control panel, the weight editor and the animation of the result.

Class constants

Constant	Description
`ANIM_PASO_MS`, `ANIM_FINAL_MS`, `ANIM_PAUSA_MS`	Timing of the animation (milliseconds): time between two lit nodes, time the complete path stays highlighted, and pause before the loop restarts.
`COLOR_BASE_ARISTA`, `COLOR_ARISTA_ILUMINADA`	Normal colour of an edge and colour of an edge lit during the animation.
`COLOR_NODO_ILUMINADO`, `COLOR_NODO_ACTUAL`	Fill colour of a node already visited by the animation and outline colour of the node currently being lit.
`COLOR_FINAL`, `COLOR_FINAL_NODO`	Colours used to highlight the complete shortest path at the end of each animation cycle.

Attributes

Attribute	Description
`filas (N), columnas (M)`	Dimensions of the grid entered by the user.
`grafo`	Instance of Grafo that holds the model.
`nodo_inicio, nodo_fin`	Start node and final node selected by the user (None while not selected).
`modo_clic`	tk.StringVar with the active click mode: "inicio", "fin" or "editar".
`camino_actual`, `costo_actual`	Last shortest path (list of nodes) and its total weight, as returned by dijkstra().
`_anim_job, _anim_paso`	Identifier of the pending after() call and number of nodes already lit in the current animation cycle.
`_redibujo_pendiente`	Identifier of the delayed redraw, used to avoid redrawing too often while the window is resized.
`canvas`	tk.Canvas where nodes, arrows and weights are drawn.
`panel_control`, `frame_edicion`	Right-side panel with controls and the dynamic frame where the weights of the selected node are edited.
`lbl_ruta, lbl_costo, lbl_nodo_sel`	Labels that show the path, the total cost and the selected node.

Methods

Method	Description
`__init__()`	Creates the window, initializes the attributes and shows the initial screen.

Method	Description
`_ajustar_ventana(espacio)`	Sizes the window to the screen leaving a margin on every side.
`_crear_pantalla_inicial()`	Builds the initial screen where the user enters the number of rows (N) and columns (M).
`_inicializar_grafo()`	Validates N and M, creates one node per grid cell (labelled from 1, row by row) and the directed connections in four directions (right, left, down, up) with random weights, then opens the main interface.
`_crear_interfaz_principal()`	Creates the canvas and the control panel (click modes, route and cost labels, weight editor).
`_calcular_celdas(w, h, margin)`	Calculates the size of a grid cell from the canvas size and the number of rows and columns.
`_recalcular_dijkstra()`	Calls Grafo.dijkstra() if both the start and end nodes are defined; otherwise clears the current path.
`redibujar_grafo()`	Clears the canvas, recalculates the shortest path and draws every arrow with its weight and every node (green = start, red = end). Updates the route and cost labels and starts the animation.
`_on_canvas_resize(event), _redibujo_diferido()`	Redraw the graph when the canvas size changes.
`_on_canvas_click(event)`	Detects which node was clicked and, depending on the active mode, sets it as start, as end or opens its weight editor.
`_definir_inicio(nodo), _definir_fin(nodo)`	Set the start / end node. If the chosen node was already the other one, both are swapped.
`_avanzar_modo()`	Changes the click mode to ask for whatever is missing (start, then end, then edit weights).
`_limpiar_inicio_fin()`	Removes the start and end nodes so they can be chosen again.
`_mostrar_editor_nodo(nodo)`	Generates one input field per outgoing connection of the selected node and the “Actualizar Pesos” button, which validates the values, updates the graph and recalculates the path.
`_iniciar_animacion(), _detener_animacion(), _animar_tick()`	Start, stop and execute one step of the animation loop.
`_iluminar_nodo(k), _pintar_nodo(nodo, actual)`	Light the k-th node of the path and the edge that reaches it.
`_resaltar_camino_completo(), _limpiar_iluminacion()`	Highlight the whole path with the final colour / return it to its neutral look to restart the loop.

3.4 Other data structures used

- **Dictionaries** (nodos, adyacencia, distancias, predecesores, coords): constant-time access by label or node.
- **Priority queue** (pq, a list managed with heapq): returns the node with the lowest distance in O(log n).
- **Set of tuples** (aristas_camino): contains the pairs (origin, destination) that belong to the shortest path, so those edges can be tagged and animated.
- **List** (camino): ordered sequence of nodes of the resulting path.

4. Instructions to graph creation

Graphs are created visually through the graphical interface. The graph is always a grid of N rows by M columns:

1. Start the program. The first screen asks for the size of the grid. Type the number of **rows (N)** and the number of **columns (M)**. The grid must contain at least 2 nodes.
2. Press **Generar Grafo** (or press Enter in the last field). The program creates one node for each cell of the grid. Each node has a numeric label, assigned from 1 row by row (left to right, top to bottom).
3. Each node is connected with its neighbours up, down, left and right. Every connection is a directed arrow, so between two neighbours there are two arrows, one in each direction. Each arrow receives a random weight between 1 and 9, which is displayed on top of it.
4. In the right panel choose the click mode: **Definir Inicio (verde)** and click a node to set the start node (it turns green); **Definir Fin (rojo)** and click a node to set the end node (it turns red). They can be changed as many times as needed, and **Limpiar Inicio y Fin** removes both.
5. To change a weight, select **Editar pesos** and click a node. The panel shows one field for each outgoing connection (“Hacia Nodo X”). Type the new integer values (zero or greater) and press **Actualizar Pesos**. The graph is redrawn and the shortest path is recalculated automatically.
6. As soon as both the start and the end nodes are defined, the algorithm runs automatically and the result is shown: the shortest path and its total cost appear in the right panel, and the path is animated on the grid.

5. Instructions to run the code

Requirements

- Python 3.8 or newer.
- Tkinter. It is included in the standard Python installers for Windows and macOS. On Linux it may need to be installed separately (for example, sudo apt install python3-tk).
- No external libraries are needed: random, heapq and math are part of Python’s standard library.

Steps

1. Unzip the delivered file and open a terminal in the folder that contains the Python file.
2. Run the program with: `python dijkstra.py` (use python3 on Linux/macOS if needed, and replace dijkstra.py with the actual name of the file).
3. A window will open with the initial screen. Follow the instructions of section 4 to create a graph and obtain the shortest path.

6. Program execution

The following images show the execution of the program, the creation of a graph and the result of the algorithm.

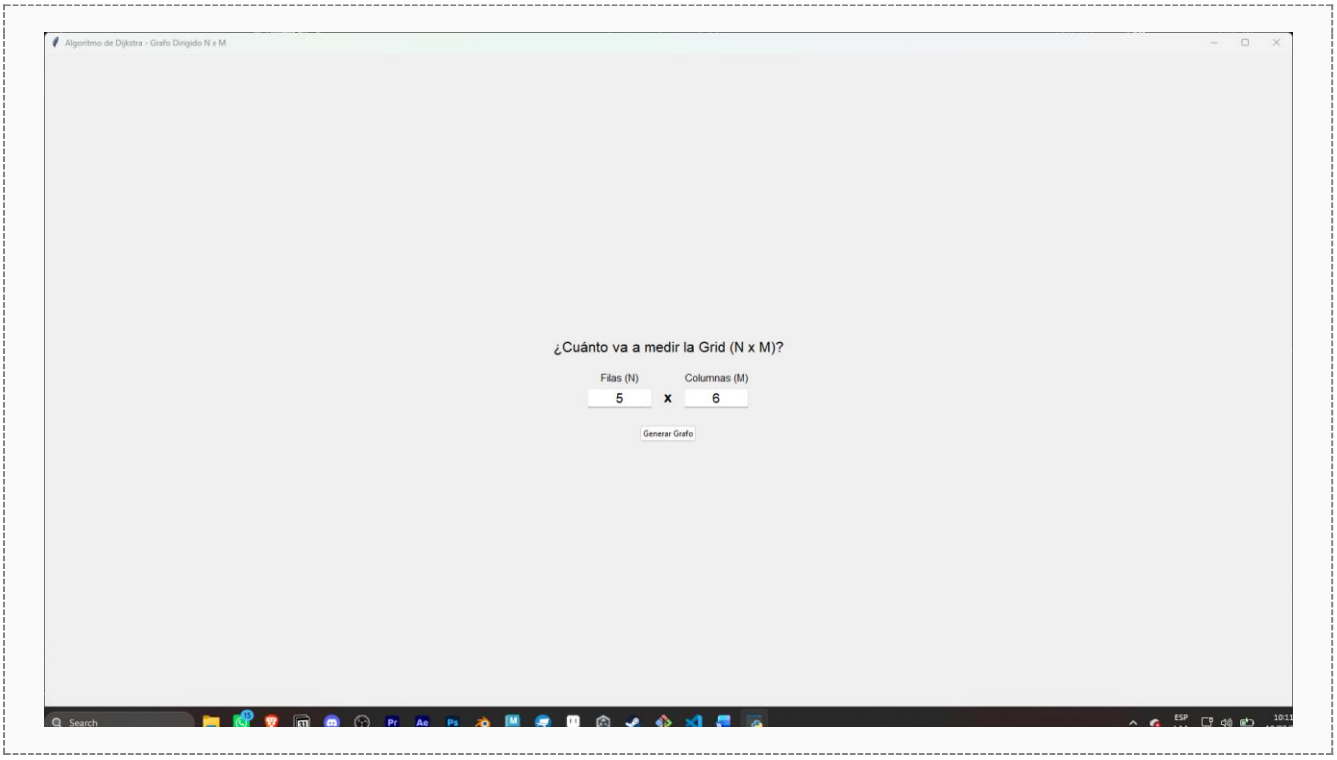


Figure 1. *Initial screen: the user enters the size of the grid (N x M).*

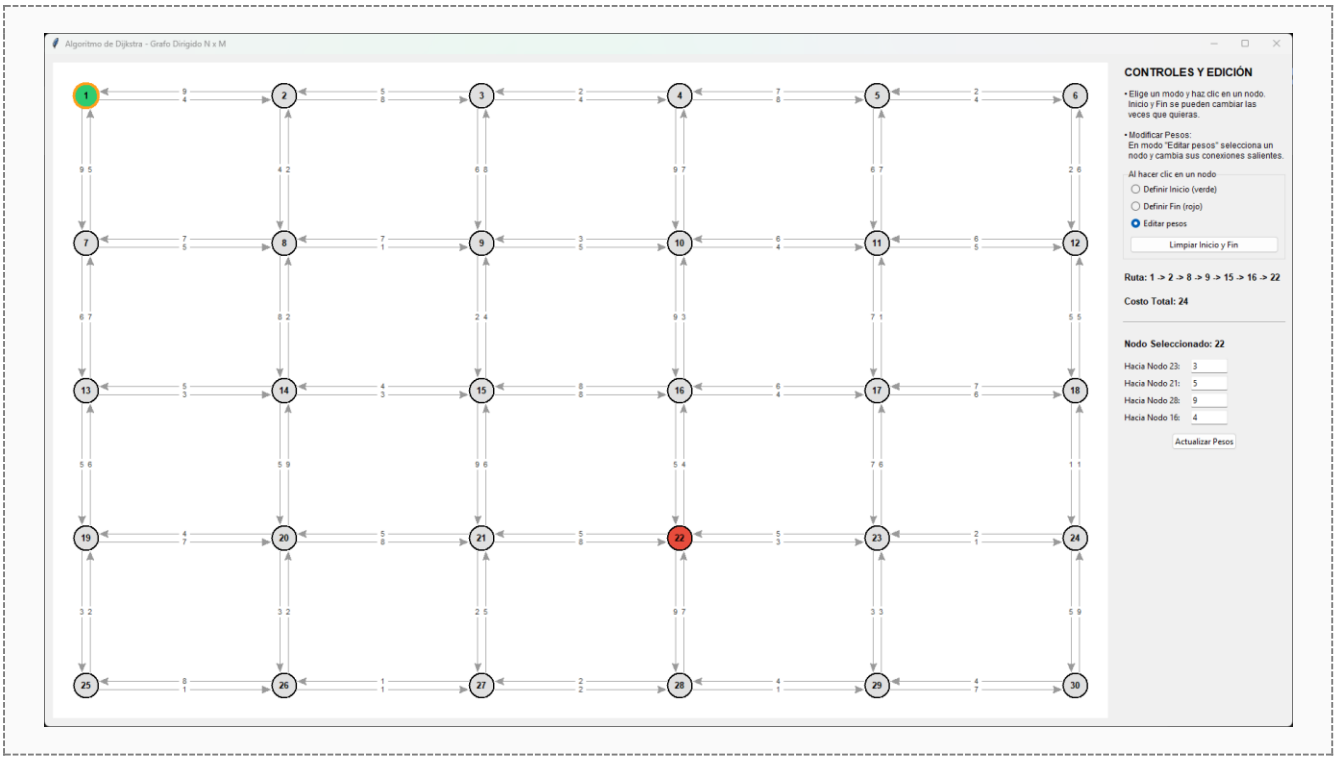


Figure 2. *Generated graph with its labelled nodes and weighted directed edges; start node (green) and end node (red) selected.*

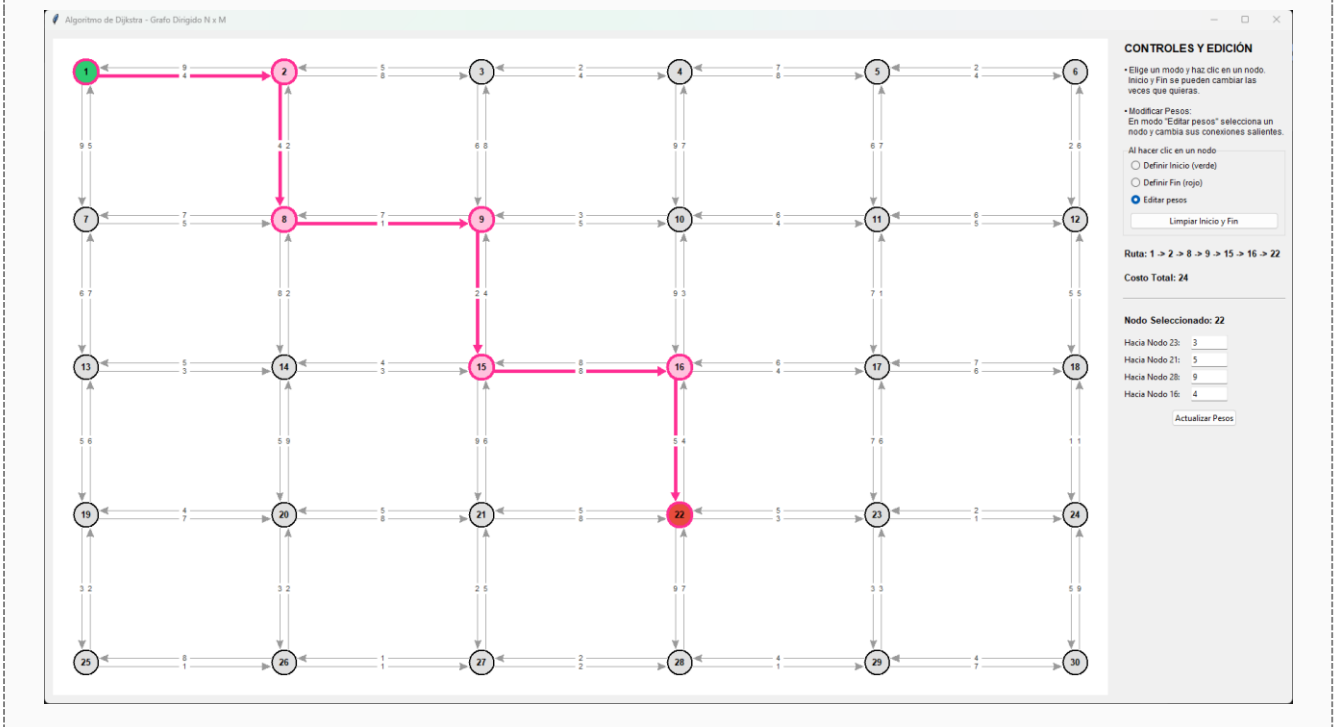


Figure 3. *Result of the algorithm: shortest path highlighted, sequence of nodes and total cost displayed.*

7. Conclusions

Uriel Arturo Monarrez Cervantes

I was in charge of the functionality of the program: the graph model and the implementation of Dijkstra's algorithm. The main challenge was to make the algorithm correct and efficient at the same time. Using a priority queue with heapq avoided searching the whole list of nodes at every step, and keeping a dictionary of predecessors made it simple to rebuild the final path. I also had to handle details that are easy to overlook, such as outdated entries in the queue, nodes that cannot be reached, and rejecting negative weights, since the algorithm is only valid with non-negative costs. Separating the logic into the Nodo and Grafo classes let the interface use it without knowing how it works inside. This project showed me that a greedy algorithm can guarantee an optimal result when its conditions are respected, and that testing it with different weights is the best way

Jaime Rincón Burboa

My responsibility was the graphical interface. Building it with Tkinter taught me that an algorithm becomes much easier to understand when it can be seen and manipulated. The biggest challenge was tracking al the things that needed to be done and connect the logic of the code with the visualization. I had to calculate the cell size from the window, shift opposite arrows so they do not overlap, and redraw the graph whenever the window is resized. I also designed the click modes for choosing the start and end nodes and the panel for editing weights, so the shortest path is recalculated automatically after each change. The program worked perfectly but i wanted to add the animation of the path helps the user follow the route from the start to the end node that helps to follow the path.

Omar Alejandro Santos Ibarra

My part of the project was the documentation, and writing the report forced me to understand the whole program, not just my own piece. To explain Dijkstra's algorithm clearly I had to follow every step in the code: how the priority queue always returns the closest node, why a node's distance is final once it is processed, and how outdated queue entries are skipped instead of removed. Relating each element of the algorithm to a specific line of the implementation made the theory much easier to retain. I also learned that documenting a project is a way of checking it: describing every class, attribute and method showed how well the model, the algorithm and the interface were separated. Finally, seeing the shortest path change right after editing a weight made me appreciate how directly this algorithm applies to pathfinding in videogames, where enemies and characters need a good route across a map in real time.